



# Artificial Intelligent LAB -8

## Genetic Algorithm (GA)

---

### What is Genetic Algorithm (GA)?

Genetic Algorithm (GA) is a metaheuristic inspired by **Darwin's theory of natural evolution**. It is used to find optimal or near-optimal solutions to complex problems where traditional optimization techniques may fail or be inefficient.

---

### Core Concepts of GA

| Concept                          | Description                                                            |
|----------------------------------|------------------------------------------------------------------------|
| <b>Population</b>                | A set of possible solutions (individuals or chromosomes)               |
| <b>Chromosome</b>                | A single solution, often represented as a binary or real-valued string |
| <b>Gene</b>                      | A part of a chromosome (e.g., variable or parameter)                   |
| <b>Fitness Function</b>          | A function that evaluates how good a solution is                       |
| <b>Selection</b>                 | Choose parents based on fitness (e.g., roulette wheel, tournament)     |
| <b>Crossover (Recombination)</b> | Combine two parents to produce offspring                               |
| <b>Mutation</b>                  | Randomly alter genes to maintain diversity                             |
| <b>Elitism</b>                   | Preserve the best solutions from one generation to the next            |

---

### GA Workflow Steps:

1. **Initialize population** randomly.
2. **Evaluate fitness** of each individual.
3. **Select parents** based on fitness.
4. **Apply crossover and mutation** to produce new population.
5. **Replace old population** with new generation.

6. **Repeat** steps 2–5 until convergence or max generations.

CODE: 1

**Maximize  $f(x) = x * \sin(10\pi x) + 1$  for  $x \in [0, 1]$**

**Objective:**

We want to find the value of  $x$  in  $[0,1]$  that maximizes this non-linear function.

---

**MATLAB Code with Step-by-Step Explanation**

clc; clear; close all;

%% Problem Definition

% Maximize  $f(x) = x * \sin(10\pi x) + 1$  over  $x \in [0, 1]$

% We use a real-coded GA (continuous values for  $x$ )

popSize = 20; % Number of individuals (solutions)

chromLength = 1; % One decision variable ( $x$ )

maxGen = 50; % Number of generations (iterations)

crossRate = 0.8; % Crossover rate (80% of individuals undergo crossover)

mutRate = 0.1; % Mutation rate (10%)

%% 🧬 Step 1: Initialize Population

% Each individual is a random number between 0 and 1

pop = rand(popSize, chromLength); % Random values of  $x \in [0, 1]$

%% Step 2: Define Fitness Function

% The higher the function value, the fitter the individual

fitnessFunc = @(x) x .\* sin(10 \* pi \* x) + 1;

% Track best fitness over generations

bestFitness = zeros(maxGen, 1);

%% Step 3: Run the GA for multiple generations

for gen = 1:maxGen

    % Step 3.1: Calculate fitness for all individuals

    fitness = fitnessFunc(pop); % Evaluate the objective for all  $x$  values

    %% Step 3.2: Selection (Roulette Wheel)

    % The fitter an individual, the more likely it will be selected

    prob = fitness / sum(fitness); % Normalize probabilities

    cumProb = cumsum(prob); % Cumulative probability

    newPop = zeros(size(pop)); % New population (empty)

    for i = 1:popSize

        r = rand;

        idx = find(cumProb >= r, 1, 'first'); % Select individual

        newPop(i,:) = pop(idx,:);

    end

    %% Step 3.3: Crossover (Arithmetic Crossover)

```

for i = 1:2:popSize
    if i+1 <= popSize && rand < crossRate
        alpha = rand; % Blend factor
        % Blend the two parents to create two children
        temp1 = alpha * newPop(i,:) + (1 - alpha) * newPop(i+1,:);
        temp2 = alpha * newPop(i+1,:) + (1 - alpha) * newPop(i,:);
        newPop(i,:) = temp1;
        newPop(i+1,:) = temp2;
    end
end

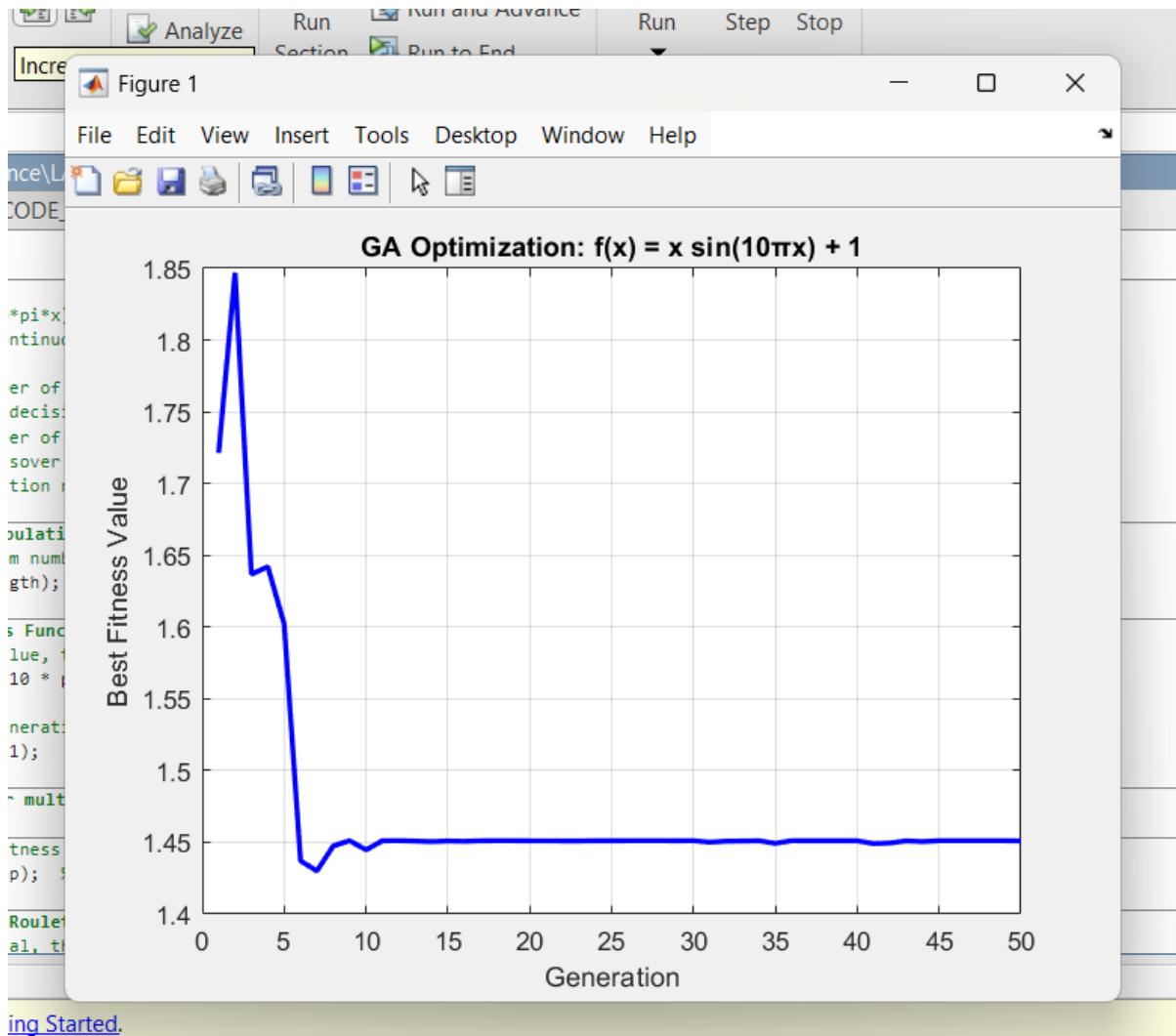
%% Step 3.4: Mutation
% Slightly change a few genes to maintain diversity
for i = 1:popSize
    if rand < mutRate
        mutationValue = 0.1 * (rand - 0.5); % Small change
        newPop(i,:) = newPop(i,:) + mutationValue;
        % Keep the value within [0,1]
        newPop(i,:) = max(0, min(1, newPop(i,:)));
    end
end

%% Step 3.5: Replace population
pop = newPop;

%% Step 3.6: Record best fitness value
bestFitness(gen) = max(fitness);
end

%% Plot the Result
figure;
plot(1:maxGen, bestFitness, 'b-', 'LineWidth', 2);
xlabel('Generation');
ylabel('Best Fitness Value');
title('GA Optimization:  $f(x) = x \sin(10\pi x) + 1$ ');
grid on;

```



CODE: 2

## Minimize Rosenbrock Function

**Problem:**

Minimize the **Rosenbrock function**:

$$f(x,y) = (1-x)^2 + 100 \cdot (y - x^2)^2$$

for  $x, y \in [-2, 2]$

clc; clear; close all;

%% Objective: Minimize the Rosenbrock function

popSize = 30;

dims = 2; % Two variables: x and y

pop = -2 + 4 \* rand(popSize, dims); % Range: [-2, 2]

maxGen = 100;

mutRate = 0.1;

fitnessFunc = @(v) (1 - v(:,1)).^2 + 100\*(v(:,2) - v(:,1).^2).^2;

```

bestFitness = zeros(maxGen, 1);

for gen = 1:maxGen
    fitness = fitnessFunc(pop);    % Lower is better

    %% Selection: Roulette Wheel (invert fitness for minimization)
    invFitness = 1 ./ (fitness + 1e-6);
    prob = invFitness / sum(invFitness);
    cumProb = cumsum(prob);
    newPop = zeros(size(pop));
    for i = 1:popSize
        r = rand;
        idx = find(cumProb >= r, 1, 'first');
        newPop(i,:) = pop(idx,:);
    end

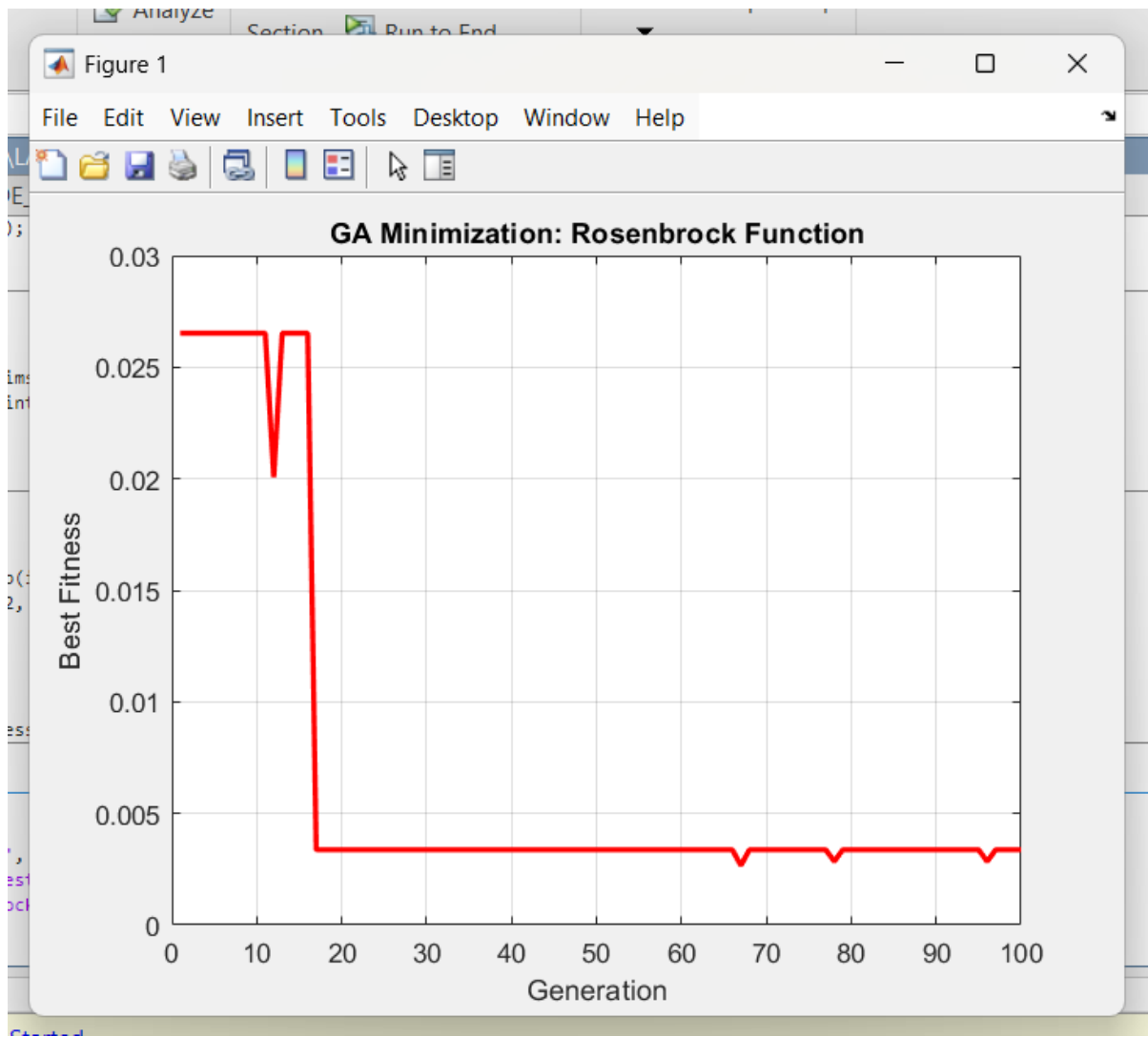
    %% Crossover: Single-point
    for i = 1:2:popSize
        if i+1 <= popSize
            point = randi([1, dims]);
            newPop([i, i+1], point:end) = newPop([i+1, i], point:end);
        end
    end

    %% Mutation
    for i = 1:popSize
        if rand < mutRate
            newPop(i,:) = newPop(i,:) + 0.1 * randn(1, dims);
            newPop(i,:) = max(-2, min(2, newPop(i,:))); % Keep in range
        end
    end

    pop = newPop;
    bestFitness(gen) = min(fitnessFunc(pop));
end

%% Plotting
figure;
plot(1:maxGen, bestFitness, 'r-', 'LineWidth', 2);
xlabel('Generation'); ylabel('Best Fitness');
title('GA Minimization: Rosenbrock Function');
grid on;

```



CODE: 3

## Optimize Weights for Linear Regression

### Problem:

Fit  $y = w_1 * x + w_2$  using GA to minimize MSE.

```
clc; clear; close all;
```

```
%% Simulated dataset
```

```
x = linspace(0, 10, 100)';
```

```
true_w = [2.5, 5]; % True slope and intercept
```

```
y = true_w(1)*x + true_w(2) + randn(size(x)); % Add noise
```

```
%% GA Settings
```

```
popSize = 30;
```

```
dims = 2; % w1 and w2
```

```
pop = -10 + 20 * rand(popSize, dims); % [-10,10] range
```

```
maxGen = 50;
```

```
mutRate = 0.1;
```

```

fitnessFunc = @(w) mean((y - (w(1)*x + w(2))).^2); % MSE

bestFitness = zeros(maxGen, 1);

for gen = 1:maxGen
    fitness = zeros(popSize, 1);
    for i = 1:popSize
        fitness(i) = fitnessFunc(pop(i,:));
    end

    %% Selection
    invFitness = 1 ./ (fitness + 1e-6);
    prob = invFitness / sum(invFitness);
    cumProb = cumsum(prob);
    newPop = zeros(size(pop));
    for i = 1:popSize
        r = rand;
        idx = find(cumProb >= r, 1);
        newPop(i,:) = pop(idx,:);
    end

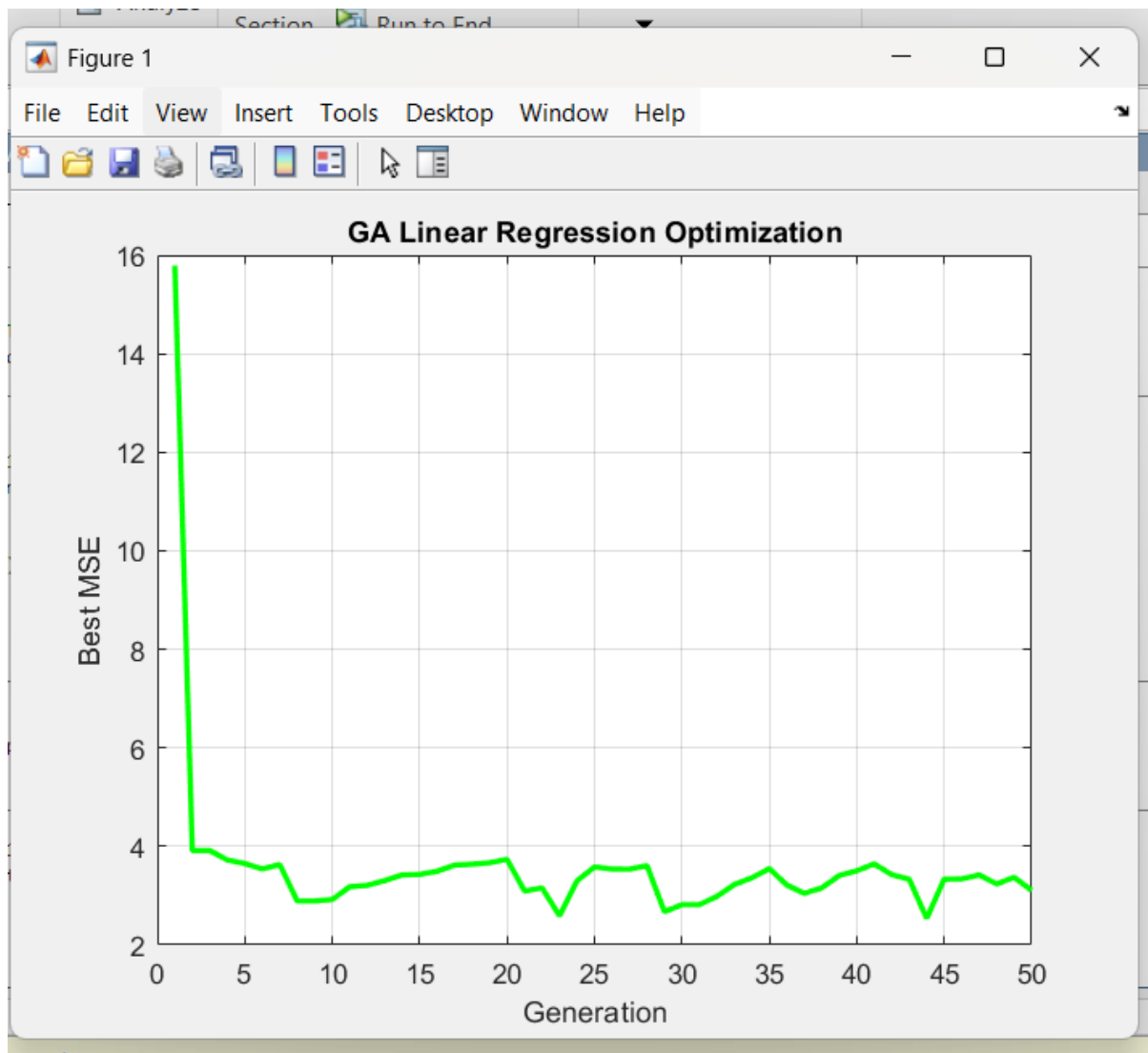
    %% Crossover
    for i = 1:2:popSize
        if i+1 <= popSize
            alpha = rand;
            newPop(i,:) = alpha * newPop(i,:) + (1-alpha) * newPop(i+1,:);
            newPop(i+1,:) = alpha * newPop(i+1,:) + (1-alpha) * newPop(i,:);
        end
    end

    %% Mutation
    for i = 1:popSize
        if rand < mutRate
            newPop(i,:) = newPop(i,:) + 0.5 * randn(1, dims);
        end
    end

    pop = newPop;
    bestFitness(gen) = min(fitness);
end

%% Plot
figure;
plot(1:maxGen, bestFitness, 'g-', 'LineWidth', 2);
xlabel('Generation'); ylabel('Best MSE');
title('GA Linear Regression Optimization');
grid on;

```



CODE: 4

## Knapsack Problem (0–1)

### Problem:

Maximize value with weight constraint using binary GA.

clc; clear;

%% Items (weights and values)

weights = [2 3 4 5];

values = [3 4 5 8];

capacity = 5;

popSize = 20;

chromLength = length(weights); % 4 items

pop = round(rand(popSize, chromLength)); % Binary representation

maxGen = 50;

mutRate = 0.05;



```

bestFitness = zeros(maxGen, 1);

for gen = 1:maxGen
    fitness = zeros(popSize, 1);
    for i = 1:popSize
        totalWeight = sum(pop(i,:) .* weights);
        totalValue = sum(pop(i,:) .* values);
        if totalWeight <= capacity
            fitness(i) = totalValue;
        else
            fitness(i) = 0; % Penalize overweight
        end
    end

    %% Selection
    prob = fitness / sum(fitness + eps);
    cumProb = cumsum(prob);
    newPop = zeros(size(pop));
    for i = 1:popSize
        r = rand;
        idx = find(cumProb >= r, 1);
        newPop(i,:) = pop(idx,:);
    end

    %% Crossover
    for i = 1:2:popSize
        if i+1 <= popSize
            point = randi(chromLength-1);
            newPop([i, i+1], point+1:end) = newPop([i+1, i], point+1:end);
        end
    end

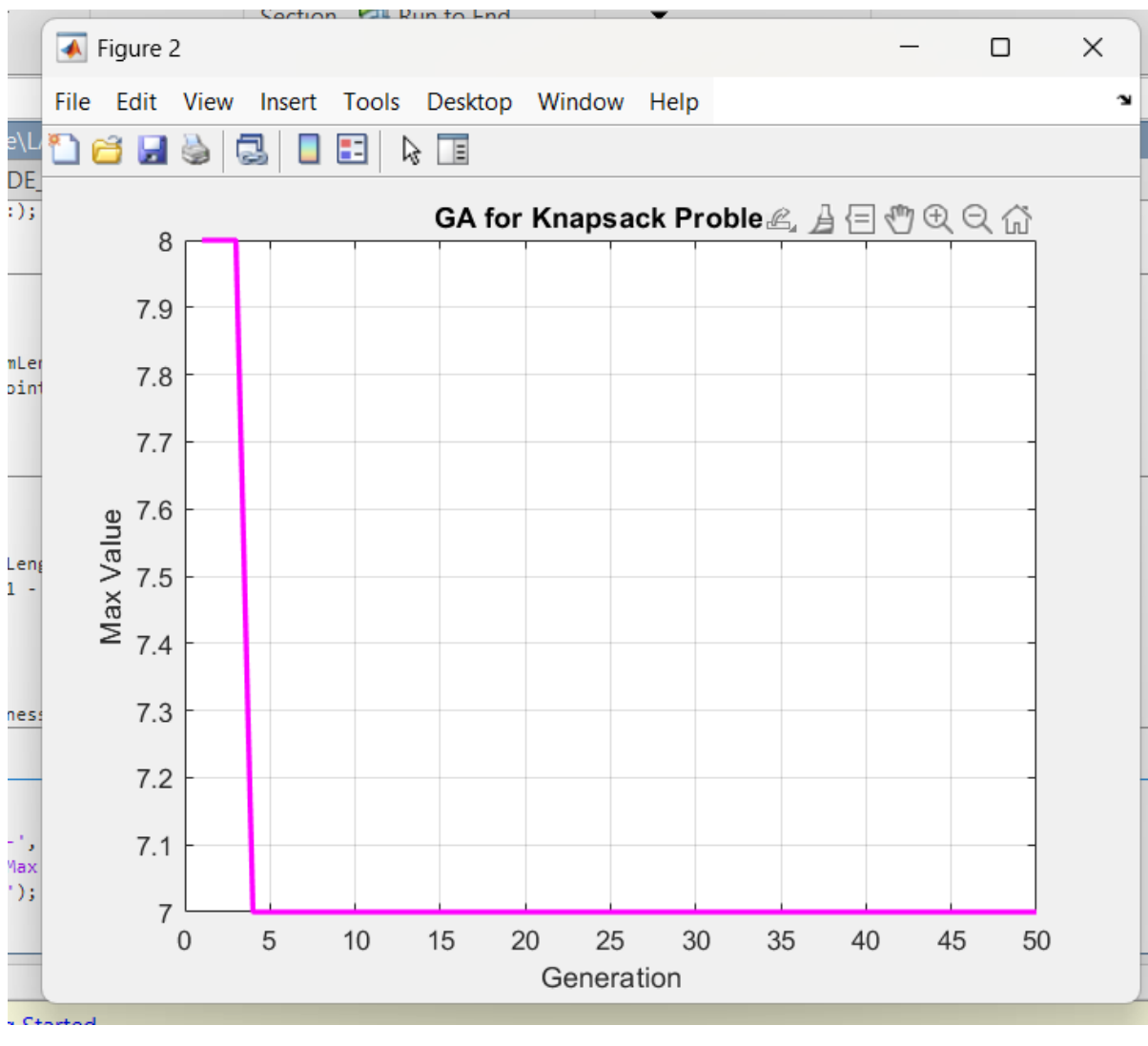
    %% Mutation
    for i = 1:popSize
        if rand < mutRate
            gene = randi(chromLength);
            newPop(i, gene) = 1 - newPop(i, gene); % Flip bit
        end
    end

    pop = newPop;
    bestFitness(gen) = max(fitness);
end

%% Plot
figure;
plot(1:maxGen, bestFitness, 'm-', 'LineWidth', 2);
xlabel('Generation'); ylabel('Max Value');
title('GA for Knapsack Problem');

```

grid on;



## CODE: 5

### Find Roots of a Polynomial

#### Problem:

Find the minimum of  $f(x) = (x-3)^2 (x+2)^2$  in range  $[-5, 5]$ .

clc; clear; close all;

%% Function with roots at -2 and 3

fitnessFunc = @(x) (x - 3).^2 .\* (x + 2).^2;

popSize = 30;

maxGen = 100;

pop = -5 + 10 \* rand(popSize, 1); % Range: [-5, 5]

mutRate = 0.1;

bestFitness = zeros(maxGen, 1);

```

for gen = 1:maxGen
    fitness = -fitnessFunc(pop); % Maximize negative of function (minimize original)

    %% Selection
    prob = fitness - min(fitness) + eps;
    prob = prob / sum(prob);
    cumProb = cumsum(prob);
    newPop = zeros(size(pop));
    for i = 1:popSize
        r = rand;
        idx = find(cumProb >= r, 1);
        newPop(i) = pop(idx);
    end

    %% Crossover
    for i = 1:2:popSize
        if i+1 <= popSize
            alpha = rand;
            newPop(i) = alpha * newPop(i) + (1-alpha) * newPop(i+1);
            newPop(i+1) = alpha * newPop(i+1) + (1-alpha) * newPop(i);
        end
    end

    %% Mutation
    for i = 1:popSize
        if rand < mutRate
            newPop(i) = newPop(i) + 0.5 * randn;
            newPop(i) = max(-5, min(5, newPop(i)));
        end
    end

    pop = newPop;
    bestFitness(gen) = min(fitnessFunc(pop));
end

%% Plot
figure;
plot(1:maxGen, bestFitness, 'k-', 'LineWidth', 2);
xlabel('Generation'); ylabel('Best f(x)');
title('GA: Find Minimum of (x-3)^2(x+2)^2');

```

grid on;

